

AtomicCAS

- Syntax: `oldval = atomicCAS(&var, x, y);`
- **Typical usecases:**
 - *Locks*: critical section processing
 - *Single*: Only one arbitrary thread executes the block.
 - Other atomic *variants*

Classwork: Implement *lock* with *atomicCAS*.

Locks

T1

```
while(1)
{
    want1 = True;
    while(want2 == True) {
        Do nothing
    }
    //Access critical section
    want1 = False;
}
```

T2

```
while(1)
{
    want2 = True;
    while(want1 == True) {
        Do nothing
    }
    //Access critical section
    want2 = False;
}
```

Lock using atomicCAS

Does this work?

```
atomicCAS(&lockvar, 0, 1);
```

Does not ensure mutual exclusion.

Then how about

```
if (atomicCAS(&lockvar, 0, 1) == 0)  
    // critical section
```

Does not block other threads.

Make the above code blocking.

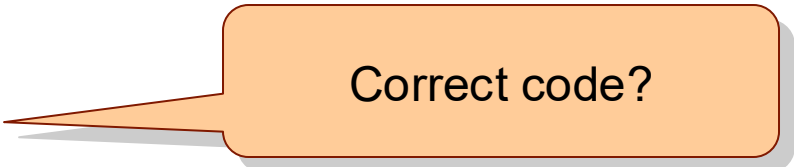
```
do {  
    old = atomicCAS(&lockvar, 0, 1);  
} while (old != 0);
```

Correct code?

Lock using atomicCAS

- The code works on CPU.
- It also works on GPU across warps.
- But it hangs for threads belonging to the same warp.
 - When one warp-thread acquires the lock, it waits for other warp-threads to reach the instruction just after the **do-while**.
 - Other warp-threads await this successful thread in the **do-while**.

```
do {  
    old = atomicCAS(&lockvar, 0, 1);  
} while (old != 0);
```



Correct code?

Lock using atomicCAS

```
do {  
    old = atomicCAS(&lockvar, 0, 1);  
} while (old != 0);  
  
// critical section  
  
lockvar = 0; // unlock
```

On CPU

```
do {  
    old = atomicCAS(&lockvar, 0, 1);  
    if (old == 0) {  
        // critical section  
        lockvar = 0; // unlock  
    }  
} while (old != 0);
```

On GPU

Classwork: Implement *single* with *atomicCAS*.

Single using atomicCAS

```
if (atomicCAS(&lockvar, 0, 1) == 0)
```

```
    // single section
```

Important not to set lockvar to 0 at the end of the single section.

What is the output?

```
#include <stdio.h>
#include <cuda.h>

__global__ void k1(int *gg) {
    int old = atomicCAS(gg, 0, threadIdx.x + 1);
    if (old == 0) {
        printf("Thread %d succeeded 1.\n", threadIdx.x);
    }
    old = atomicCAS(gg, 0, threadIdx.x + 1);
    if (old == 0) {
        printf("Thread %d succeeded 2.\n", threadIdx.x);
    }
    old = atomicCAS(gg, threadIdx.x, -1);
    if (old == threadIdx.x) {
        printf("Thread %d succeeded 3.\n", threadIdx.x);
    }
}

int main() {
    int *gg;
    cudaMalloc(&gg, sizeof(int));
    cudaMemset(&gg, 0, sizeof(int));
    k1<<<2, 32>>>(gg);
    cudaDeviceSynchronize();

    return 0;
}
```

- Some thread out of 64 updates gg to its threadid+1.
- Warp threads do not execute atomics together! That is also done sequentially.
- Irrespective of which thread executes the first atomicCAS, no thread would see gg to be 0. Hence second printf is not executed at all.
- If gg was updated by some thread 0..30, then the corresponding thread with id 1..31 from either of the blocks would update gg to -1, and execute the third printf.
- Otherwise, no one would update gg to -1, and no one would execute the third printf.
- On most executions, you would see the output to be that thread 0 would execute the first printf, and thread 1 would execute the third printf.

Classwork

- Each thread adds elements to a worklist.
 - e.g., next set of nodes to be processed in SSSP.
 - worklist is implemented as an array.
- Initially, assume that each thread adds exactly K elements.
- Later, relax the constraint.

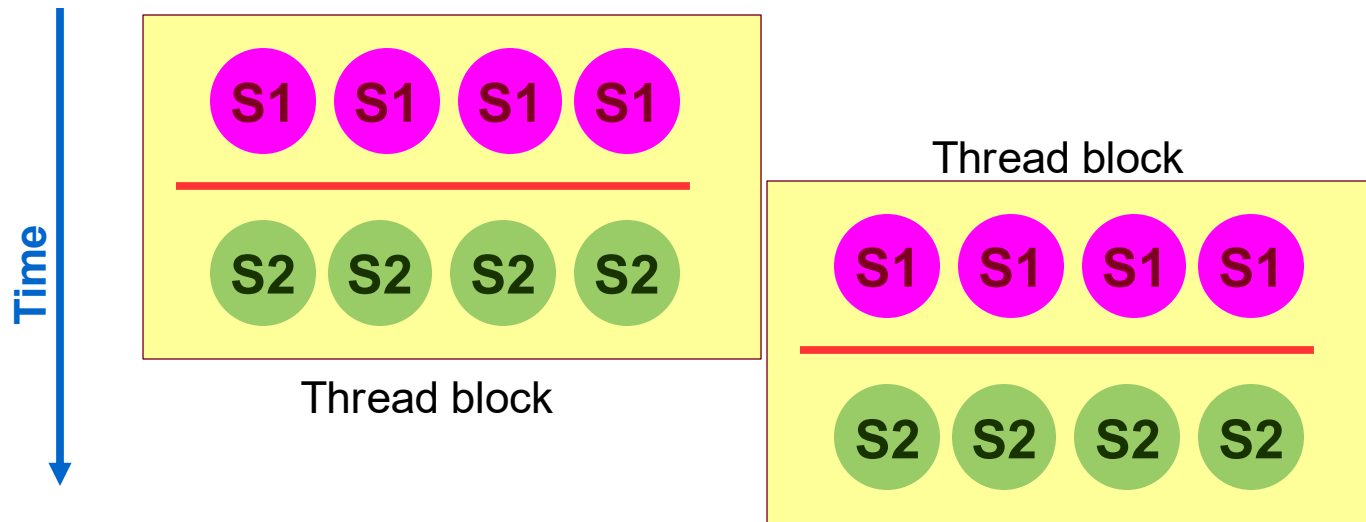
atomic-worklist.cu

Barriers

- A barrier is a program point where all threads need to reach before any thread can proceed.
- End of kernel is an implicit barrier for all GPU threads (**global barrier**).
- ~~There is no explicit global barrier supported in CUDA.~~ *grid.sync()* is now supported (from CUDA 9).
- Threads in a thread-block can synchronize using **`_syncthreads()`**.
- How about barrier within warp-threads?

Barriers

```
__global__ void dkernel(unsigned *vector, unsigned vectorsize) {  
    unsigned id = blockIdx.x * blockDim.x + threadIdx.x;  
    vector[id] = id; S1  
    __syncthreads();  
    if (id < vectorsize - 1 && vector[id + 1] != id + 1)  
        printf("syncthreads does not work.\n"); S2  
}
```



Barriers

- `__syncthreads()` is not only about control synchronization, it also has data synchronization mechanism.
- It performs a **memory fence** operation.
 - A memory fence ensures that the writes from a thread are made visible to other threads.
 - `__syncthreads()` executes a fence for all the block-threads.
- There is a separate `__threadfence_block()` instruction also. Then, there is `__threadfence()`.
- *[In general]* A fence does not ensure that other thread will read the updated value.
 - This can happen due to caching.
 - The other thread needs to use `volatile` data.
- *[In CUDA]* a fence applies to both read and write.

Classwork

- Write a CUDA kernel to find **maximum** over a set of elements, and then let thread 0 print the value in the same kernel.
- Each thread is given `work[id]` amount of work. Find average work per thread and if a thread's work is above average + K , push extra work to a worklist.
 - This is useful for **load-balancing**.
 - Also called **work-donation**.

Taxonomy of Synchronization Primitives

Primitive	Control-sync	Data-sync
__syncthreads __syncwarp	Block Warp	Block Warp
atomic	--	Block for shared All for global
__threadfence_block	--	block
__threadfence	--	All
Global barrier	All	All
while loop	Customizable	— (but not useful without data-synchronization)
volatile	--	All

We looked at How to Handle Data Races

- A datarace occurs if **all** of the following hold:
 1. Multiple threads
 2. Common memory location
 3. At least one write
 4. Concurrent execution
- Ways to remove datarace:
 1. Execute sequentially
 2. Privatization / Data replication
 3. Separating reads and writes by a barrier
 4. Mutual exclusion